HIGH-LEVEL DESIGN DOCUMENT

RAG-Based Customer Support Assistant with HITL

Property	Value
Project Name	RAG-Based Customer Support Assistant
Document Type	High-Level Design (HLD)
Version	1.0
Date	April 2026
Author	Abhishek Sambhaji Bhosale
Intern ID	IN226053602
Organization	Innomatics Research Labs

1. System Overview

1.1 Problem Statement

Customer support teams face significant scalability challenges when dealing with high volumes of repetitive queries. Traditional rule-based chatbots lack contextual understanding and fail to handle nuanced questions, while fully human-operated support is expensive and slow to respond, particularly during peak hours.

Key challenges include:

- Response inconsistency across different human agents
- High operational costs — 50+ agents for 10,000+ daily queries
- 5-minute average response time leading to poor user experience
- Inability to scale without proportional headcount increase

1.2 Proposed Solution

A Retrieval-Augmented Generation (RAG) system that:

- Answers routine queries automatically using a PDF-based knowledge base
- Uses vector similarity search to retrieve contextually relevant information
- Applies GPT-3.5-turbo for natural language response generation
- Escalates complex or low-confidence queries to human agents via HITL workflow
- Maintains a confidence scoring mechanism to gate direct vs. escalated responses

1.3 Scope

In Scope

- PDF document ingestion and semantic chunking
- Vector embedding storage using ChromaDB
- LLM-based response generation with GPT-3.5-turbo
- Conditional escalation logic with 60% confidence threshold
- Human-in-the-Loop (HITL) agent dashboard via Streamlit
- Persistent escalation queue using JSON file storage

Out of Scope

- Multi-language support
- Real-time live agent chat (WebSocket)
- Voice interface (speech-to-text/TTS)
- Multi-PDF concurrent support (planned future enhancement)
- User authentication and access control

2. System Architecture

2.1 High-Level Architecture Overview

The system is organized into five distinct functional layers, each with clear responsibilities and interfaces:

Layer	Components	Responsibility
User Layer	Streamlit UI, Human Agent Dashboard	User interaction and agent review interface
Application Layer	Query Processor, LangGraph Workflow Engine	Request routing and stateful workflow management
RAG Layer	ChromaDB Retriever, OpenAI Embeddings	Semantic document retrieval
LLM Layer	GPT-3.5-turbo Generator	Natural language response generation
HITL Layer	Escalation Manager, JSON Queue	Low-confidence query routing to humans

2.2 Data Pipeline

```
PDF File
  └── PyPDFLoader (LangChain)
       └── Raw Text Pages
            └── RecursiveCharacterTextSplitter (500 chars, 50 overlap)
                 └── Text Chunks with Metadata
                      └── OpenAI text-embedding-3-small (1536 dims)
                           └── ChromaDB Vector Store (persisted to disk)
```

2.3 Query Processing Pipeline

```
User Query
  └── Intent Detection (keyword classification)
       └── LangGraph Workflow Starts
            └── [Node 1] Retrieve: Vector similarity search (k=4)
                 └── Context Assembly (top-4 chunks concatenated)
                      └── [Node 2] Generate: LLM invocation with context+query
                           └── Confidence Score Calculation (0.0 - 1.0)
                                ├── score >= 0.6 → Direct Response to User
                                └── score < 0.6  → [Node 3] Escalate to HITL Queue
```

3. Component Description

3.1 Document Ingestion Pipeline

Component	Technology	Configuration	Purpose
Document Loader	PyPDFLoader (LangChain)	Input: PDF file path	Extracts raw text from PDF pages with metadata
Text Splitter	RecursiveCharacterText Splitter	chunk_size=500, overlap=50	Splits text into semantically coherent overlapping chunks
Embedding Model	text-embedding-3-small (OpenAI)	Dimensions: 1536	Converts text chunks into dense vector representations

3.2 Storage Layer

Component	Technology	Purpose
Vector Store	ChromaDB	Stores embeddings with source metadata, supports similarity search
Persistence	Local disk (./chroma_db/)	Maintains vector index across application restarts
Escalation Queue	JSON file (escalations.json)	Stores pending and resolved HITL tickets with full context

3.3 Processing Layer

Component	Technology	Purpose
Workflow Engine	LangGraph (StateGraph)	Manages stateful query processing with conditional node routing
LLM Generator	GPT-3.5-turbo (OpenAI)	Generates customer support responses grounded in retrieved context
Retriever	ChromaDB similarity search	Fetches top-k=4 most relevant document chunks per query
Intent Classifier	Keyword-based Python logic	Categorizes queries into product/return/technical/general/unknown

3.4 Interface Layer

Component	Technology	Purpose
Web Chat UI	Streamlit	Interactive chat interface with message history display
Agent Dashboard	Streamlit Sidebar	View, review, and resolve pending escalation tickets
Session State	Streamlit session_state	Maintains chat history and escalation list within session

4. Technology Choices & Justification

4.1 Why ChromaDB?

Criteria	ChromaDB	Pinecone	FAISS	Decision
Setup Complexity	Low (pip install)	High (account + API)	Medium (manual config)	ChromaDB
Local Deployment	Yes	No (cloud only)	Yes	ChromaDB
Built-in Persistence	Yes	Yes (cloud)	Manual serialization	ChromaDB
Metadata Filtering	Yes	Yes	No	ChromaDB
Cost	Free	$0.10+/hour	Free	ChromaDB

4.2 Why LangGraph?

Feature	Benefit for This Project
Conditional Edges	Enables clean escalation vs. direct response routing logic
TypedDict State	Strongly typed state object flows through all nodes without ambiguity
MemorySaver	Supports conversation thread continuity across multi-turn queries
Visual Debug	Graph execution paths are traceable and explainable
Async Support	Enables non-blocking LLM and retriever calls for better performance

4.3 LLM Choice: GPT-3.5-turbo

Factor	GPT-3.5-turbo	GPT-4	Justification
Cost per query	$0.002	$0.03	15x cheaper; sufficient for support domain
Avg. Latency	~1.5 seconds	~3 seconds	Faster user experience
Context Window	16K tokens	128K tokens	16K sufficient for 4 retrieved chunks
Quality	Good	Excellent	Good enough for structured support answers

5. Scalability Considerations

5.1 Handling Large Documents

Challenge	Current Limit	Mitigation Strategy
PDF > 100 pages	Supported but slower ingestion	Increase chunk_size to 1000, use async loading
Multiple PDFs	Single PDF supported	Add document source metadata filtering in ChromaDB
Memory constraints	Entire PDF loaded to RAM	Stream chunks instead of loading all pages at once

5.2 Query Load Scaling

Component	Current Capacity	Scaling Strategy
Vector Store (ChromaDB)	Concurrent reads supported	Migrate to managed Pinecone for high-traffic production
LLM API Calls	Sequential per request	Implement request queuing with async batch processing
Web Interface (Streamlit)	Single-user process	Migrate to FastAPI + React with multiple worker processes

5.3 Latency Targets

Operation	Target P95	Current Actual	Mitigation
Document Ingestion (50 pages)	< 30 seconds	~15 seconds	Acceptable for one-time setup
Query Response (end-to-end)	< 3 seconds	~2 seconds	Cache frequent query embeddings
Vector Similarity Search (k=4)	< 500ms	~200ms	Already optimal
Escalation Resolution	< 5 minutes	Manual	Add agent push notification system

6. Security Considerations

Risk	Description	Mitigation
API Key Exposure	OpenAI API key in source code	Store in .env file; add .env to .gitignore
Sensitive Data in PDF	Customer PII in knowledge base	Encrypt ChromaDB persistence directory
Unauthorized Dashboard Access	Any user can view escalation tickets	Add authentication layer (future scope)
Prompt Injection	Malicious query manipulating LLM	System prompt constraints + input sanitization

7. Assumptions & Dependencies

7.1 Assumptions

- The input PDF contains well-formatted, extractable text (not scanned images)
- User queries are submitted in English language only
- A single PDF knowledge base is used per deployment instance
- OpenAI API service is available and the API key has sufficient quota
- The system is deployed in a trusted internal environment (no auth required)

7.2 External Dependencies

Dependency	Version	Purpose	Risk if Unavailable
Python	3.9+	Runtime environment	System cannot run
OpenAI API	Latest	Embeddings + LLM generation	Total system failure
ChromaDB	0.4.22	Vector storage and retrieval	No document search
LangGraph	0.0.20	Workflow orchestration	No stateful routing
Streamlit	1.29.0	Web UI rendering	No user interface

8. Appendix

8.1 Glossary

Term	Definition
RAG	Retrieval-Augmented Generation — AI pattern combining vector search with LLM generation
HITL	Human-in-the-Loop — a design pattern routing AI uncertainty to human reviewers
Chunk	A 500-character segment of document text produced by the text splitter
Embedding	A 1536-dimension vector representation of text produced by OpenAI's embedding model
Confidence Score	A computed float (0.0–1.0) estimating the reliability of an LLM response
k	The number of top similar document chunks retrieved per query (default: 4)
LangGraph	A framework for building multi-node, stateful LLM application workflows

8.2 References

- LangGraph Documentation: https://langchain-ai.github.io/langgraph/
- ChromaDB Documentation: https://docs.trychroma.com/
- OpenAI Embeddings Guide: https://platform.openai.com/docs/guides/embeddings
- LangChain PDF Loaders: https://python.langchain.com/docs/modules/data_connection/document_loaders/